

Linux System Calls

1. `execve()` - Execute Program

This is the syscall that actually runs a program. It replaces the current process image with a new process image. It takes the path to the executable, the arguments array, and the environment variables array.

```
# Tracing the execution of the 'ls' command

root@linux:~# strace -e execve ls
execve("/usr/bin/ls", ["ls"], 0x7ffd5a... /* 54 vars */) = 0
```

2. `openat()` / `open()` - Open a File

Opens a file and returns a non-negative integer known as a File Descriptor (FD). `openat` is a modern variant that opens a file relative to a directory file descriptor.

```
# Tracing 'cat /etc/passwd' to see file opening

root@linux:~# strace -e openat cat /etc/passwd
openat(AT_FDCWD, "/etc/passwd", O_RDONLY) = 3
/* Returns File Descriptor 3 (0=stdin, 1=stdout, 2=stderr are already
taken) */
```

3. read() - Read from a File Descriptor

Reads a specified number of bytes from an open file descriptor into a buffer. It returns the number of bytes successfully read.

```
# Tracing the read syscall on File Descriptor 3

root@linux:~# strace -e read cat /etc/passwd
read(3, "root:x:0:0:root:/root:/bin/bash
...", 131072) = 2841
/* Read 2841 bytes from the file into the buffer */
```

4. write() - Write to a File Descriptor

Writes bytes from a buffer to an open file descriptor. Writing to standard output (the terminal screen) is done by writing to file descriptor 1.

```
# Tracing 'echo "Hello"'

root@linux:~# strace -e write echo "Hello"
write(1, "Hello
", 6) = 6
/* Writes 6 characters (including newline) to stdout (FD 1) */
```

5. `close()` - Close a File Descriptor

Closes a file descriptor, releasing it back to the kernel so it can be reused by subsequent open calls.

```
# Tracing the closure of a file  
  
root@linux:~# strace -e close cat /etc/passwd  
close(3) = 0  
/* Returns 0 on success, indicating FD 3 is now closed */
```

6. `clone()` - Create a Child Process

The primary system call used by the Linux kernel to create new processes or threads. It is the modern backend for the classic `fork()` function.

```
# A bash shell spawning a new process  
  
root@linux:~# strace -e clone bash -c "ls"  
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|...|SIGCHLD) = 14592  
/* Returns the Process ID (PID) of the new child process: 14592 */
```

7. wait4() - Wait for Process to Change State

Used by a parent process to wait for a child process to terminate. It prevents child processes from becoming "zombies" by reaping their exit status.

```
# The parent shell waiting for its child command to finish

root@linux:~# strace -e wait4 bash -c "sleep 1"
wait4(-1, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 14593
/* Successfully waited for child PID 14593 to exit */
```

8. mmap() - Map Memory

Maps files or devices into memory. It is also heavily used by memory allocators (like malloc) to allocate large blocks of anonymous RAM for a process.

```
# Tracing memory allocation

root@linux:~# strace -e mmap ls
mmap(NULL, 131072, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
= 0x7f9a1b2c4000
/* Returns the starting memory address of the allocated block */
```

9. kill() - Send a Signal

Despite the name, `kill()` is used to send *any* signal to a process, not just to terminate it. Sending `SIGTERM` or `SIGKILL` is simply the most common use case.

```
# Sending a termination signal to PID 8432

root@linux:~# strace -e kill kill -9 8432
kill(8432, SIGKILL) = 0
/* Signal sent successfully (returns 0) */
```

10. exit_group() - Terminate all Threads

Terminates all threads in the current process group. This is what actually runs when a program finishes successfully (e.g., returning 0 from the `main` function).

```
# The final breath of any command

root@linux:~# strace -e exit_group echo "Done"
Done
exit_group(0) = ?
/* Process is terminated, so it never returns a value (?) back to itself */
```